DATA NORMALIZATION

Data normalization in DBMS is the process of organizing the data of a table in its simplest, most meaningful form to improve data integrity and reduce redundancy. It involves restructuring data so that it becomes less repetitive and more logically stored.

Goals of Normalization

We normalize data using various normal forms, each applied step by step, to achieve the following goals:

→ To eliminate redundancy in data

→ To ensure dependencies between data are logical

→ To improve consistency, accuracy, and ease of maintenance

The process of normalization was proposed by Edgar F. Codd and involves a series of methods to achieve a normalized database. These methods are categorized as normal forms — they are 1NF, 2NF, 3NF, BCNF (3.5NF), 4NF, 5NF and theoretically further. In practical scenarios, most designs don't go beyond 3NF or BCNF.

In this course, we'll only cover the normalization forms up to 3NF.

Let's consider the following example of student data:

Student_Id	Student_Name	Subjects	Teachers
1001	Alisha	Physics, Maths	Mr. X, Mr. Y
1002	Gunzan	Biology, Computer	Mr. Z, Mr. A
1003	Dristina	Maths, Computer	Mr. Y, Mr. A

The above table is not atomic, since it contains mutiple values in Subjects and Teachers column. This is problematic because if even one subject or teacher changes, we must rewrite the entire

WHAT IS ATOMICITY?

Atomicity refers to the concept that each field in a record should contain a single, indivisible value. If a field contains multiple values (e.g., "Physics, Maths"), it is said to violate atomicity.

field — increasing the risk of inconsistency. It also makes searching, filtering, and updating data more difficult.

To fix this, we move to First Normal Form (1NF), where the primary objective is to achieve atomicity.

First Normal Form (1NF)

To bring the data into 1NF, we split the multi-valued fields into separate rows, so that each field contains only one value.

Student_Id	Student_Name	Subjects	Teachers
1001	Alisha	Physics	Mr. X
1001	Alisha	Maths	Mr. Y
1002	Gunzan	Biology	Mr. Z
1002	Gunzan	Computer	Mr. A
1003	Dristina	Maths	Mr. Y
1003	Dristina	Computer	Mr. A

Primary Key: Student_Id + Subject

Now, every field is atomic — containing only one value. However, there is still visible redundancy, such as repeated student names. To reduce this, we move to Second Normal Form (2NF), which uses the concept of functional dependency

WHAT IS FUNCTIONAL DEPENDANCY?

A functional dependency occurs when the value of one attribute is determined by another.

For example:
Student_Id → Student_Name

This means that if we know the StudentId, we can determine the corresponding StudentName.

In our current 1NF table, each row has a composite key — both StudentId and Subject are needed to uniquely identify a row. However, StudentName depends only on StudentId. This is a case of partial dependency, which violates the rules of 2NF.

Second Normal Form (2NF)

In 2NF, we remove partial dependencies by splitting the table. Any field that depends only on part of a composite key should be moved to a separate table.

Student_Id	Student_Name
1001	Alisha
1002	Gunzan
1003	Dristina

Student_Id	Subjects	Teachers
1001	Physics	Mr. X
1001	Maths	Mr. Y
1002	Biology	Mr. Z
1002	Computer	Mr. A
1003	Maths	Mr. Y
1003	Computer	Mr. A

Why Stop Here? What's Still Wrong?

At first glance, everything looks fine — no multi-valued fields, no partial dependencies. However, if we observe carefully, we can see that Teacher is determined not by StudentId, but by Subject.

In other words:

Student_Id → Subject → Teacher

Here, Teacher is transitively dependent on StudentId.

What is a Transitive Dependency?

A transitive dependency exists when a non-key attribute depends on another non-key attribute, which in turn depends on the key.

This indirect relationship makes data maintenance harder and increases the risk of inconsistency.

Third Normal Form (3NF)

To remove the transitive dependency, we separate the Teacher information from the student-subject relationship.

Student_Id	Student_Name
1001	Physics
1001	Maths
1002	Biology

StudentId	Subject
1001	Physics
1001	Maths
1002	Biology
1002	Computer
1003	Maths
1003	Computer

Subject	Teacher
Physics	Mr. X
Maths	Mr. Y
Biology	Mr. Z
Computer	Mr. A

This normalized structure is now clean, consistent, and easier to maintain. It avoids redundancy, reduces risk of inconsistency, and ensures that each data item has its proper place.

Summarization

Normal Form	Description	What Was Fixed
0NF	Unnormalized data with multivalued fields	
1NF	Atomic values only	Removed repeating groups/multi-values
2NF	No partial dependency	Separated data that depended on part of key
3NF	No transitive dependency	Separated data that indirectly depended on key